

Phase 2 — What We Built

Industrial Operations Intelligence Platform — Orchestration & Analytics layer, explained simply.

The goal of Phase 2

Phase 1 of the platform allowed a user to upload a CSV file, map each of its columns to one of the eight universal business domains, and store the cleaned data inside a central data hub. That gave us a solid foundation, but the data simply sat there as raw records. Phase 2 builds the next layer on top of that foundation: it takes the stored data and **automatically turns it into useful analytics and machine-learning-ready features**, then displays them on a dashboard.

To do this in a professional, production-style way, Phase 2 introduces two industry-standard data-engineering tools — **Apache Airflow**, which schedules and orchestrates data pipelines, and **Apache Spark**, which processes the data at scale. Together they form the processing engine of the platform. The key idea is that the whole journey from raw data to finished analytics happens on its own, without the user having to run any manual commands or technical steps.

In one line: **upload → confirm → the system automatically computes analytics and features → the dashboard shows them**. This turns the platform from a simple storage system into a working data pipeline that produces real insights from any business's data.

What we added

Phase 2 added several new components, each with a clear job. Below is what each piece does and why it is needed. Every component was built to be permanent foundation, so later phases can build directly on top of it without rewriting anything.

- **Apache Airflow (orchestration).** Airflow is the tool that runs data pipelines on a schedule or on demand. We set it up with two pipelines (called DAGs): one that handles scheduled data ingestion, and one that runs the analytics jobs. Airflow gives us a visual interface to see each run, its status, and its logs, which is exactly how real data teams operate.
- **Apache Spark (processing engine).** Spark is a big-data engine that can process large volumes of data efficiently. We set up a Spark cluster that connects to the hub, reads the stored data, performs calculations, and writes the results back. Even though our datasets are modest in size, using Spark shows the platform is built to scale to large, enterprise-level data.
- **Two Spark jobs (the actual computation).** The first job computes analytics for each domain — totals, averages, minimums, maximums, and daily trends over time. The second job builds machine-learning features for each entity (for example, each machine): its average value, its variation, its latest value, and whether it is trending up or down. These features are the raw material for predictive models later.
- **A dedicated analytics database area.** We created three new tables to hold the computed results — domain metrics, daily trend, and entity features. These are deliberately kept separate from the raw hub data, so the original records and the computed results never mix and each stays clean and easy to manage.
- **Analytics API endpoints.** We added new backend routes so the dashboard can request the computed metrics, trends, and features whenever it needs them. This keeps the frontend and the data cleanly separated, which is good engineering practice.
- **A new Analytics dashboard page.** Finally, we built a page that displays everything: a summary, a metrics chart, a trend chart, a machine-learning feature table, and a domain dashboard that shows one chart per active domain, each chart chosen to suit that domain's data.

How the automatic flow works

The most important achievement of Phase 2 is that the analytics happen automatically, with no manual steps. When a user finishes uploading a CSV and clicks **Confirm**, the platform first saves the mapped data into the hub. This part is immediate, which is why the Review and Datasets pages show the new data right away.

At that same moment, the system quietly sends a signal to Airflow to run the analytics pipeline — but only for that one newly uploaded dataset, not for everything. Airflow then launches the two Spark jobs in the background. Spark reads the new dataset from the hub, computes its analytics and its machine-learning features, and writes the results into the analytics tables. This usually takes about one to two minutes.

Once the jobs finish, the dashboard can immediately display the results for that dataset. From the user's point of view, the experience is simple and clean: they upload a file, confirm it, wait a short moment, and the analytics appear on their own. They never open a terminal, never run a command, and never need to understand Airflow or Spark. This is exactly how a real self-service data platform should behave, and it is the core deliverable of Phase 2.

The domain dashboard

A major challenge is that every business is different, so every dataset contains a different mix of domains. A factory dataset might only have Assets, Quality, and Maintenance, while a telecom dataset might have Finance and Customers instead. The dashboard had to handle this gracefully, showing the right charts for whatever data is present and nothing for the data that is absent.

To solve this, the domain dashboard first shows all eight universal domains at the top as a status row. The domains that have data in the current dataset are marked with a tick, and the ones that do not are shown in grey with a dash. This gives an instant, clear picture of what the dataset actually contains, which is very useful for a reviewer or a manager looking at it for the first time.

Below the status row, the dashboard draws a chart only for each active domain — and it picks a chart type that suits that domain's data. For example, it uses a line chart for Assets, a bar chart for Operations, a bubble chart for Quality, a treemap for Inventory, a horizontal bar for Workforce, an area chart for Finance, and a donut chart for Customers. Domains with no data are simply hidden, so there are never any empty or confusing charts. The result is a clean, dynamic dashboard that adapts itself to any dataset the user uploads.

Why this matters (the takeaway)

Phase 2 transforms the platform from a place that simply stores data into a genuine data-engineering system. Data now flows in, gets processed automatically by Airflow and Spark, and comes out the other side as meaningful analytics and machine-learning-ready features, all visible on a live dashboard. This is the complete journey that real industrial data platforms perform every day.

Just as importantly, nothing built here is throwaway code. The analytics tables and the engineered feature tables are the direct foundation for Phase 3, where predictive machine-learning models (such as predictive maintenance) will be trained on exactly these features. Because the feature engineering is already done and running, Phase 3 can focus purely on modelling rather than data preparation.

The design also reflects mature engineering thinking. The platform supports two processing patterns: a real-time pattern that processes a single dataset instantly on upload, and a scheduled batch pattern through Airflow for bulk or overnight recomputation. Using the right pattern for the right situation — fast processing for interactive uploads, and scheduled orchestration for large or repeated jobs — is precisely

how professional data systems are built, and it gives the project a strong, defensible story for both the capstone review and technical interviews.